

Proyek Sistem Operasi

Minggu 14: Arsitektur Sistem Berkas & Crash Recovery di xv6

Novalio Daratha
Program Studi Informatika
Universitas Bengkulu

2026

Outline Minggu 14

- 1 7 Lapisan Sistem Berkas xv6
- 2 Struktur Inode & Alokasi Blok
- 3 Logging & Transaksi Atomik

Hierarki Lapisan File System

- 1 **File Descriptors** (`file.c`): Abstraksi file/pipa/alat untuk proses pengguna.
- 2 **Pathnames** (`fs.c`): Parsing nama berkas hierarkis (`/usr/bin/sh`).
- 3 **Directories** (`fs.c`): Tabel pemetaan nama file ke nomor inode (`inum`).
- 4 **Inodes** (`fs.c`): Struktur berkas individu, ukuran, atribut, dan pointer blok disk.
- 5 **Logging** (`log.c`): Transaksi atomik untuk pemulihan kegagalan sistem (*Crash Recovery*).
- 6 **Buffer Cache** (`bio.c`): Caching blok disk di RAM untuk mengurangi latensi disk I/O.
- 7 **Disk Driver** (`virtio_disk.c`): Driver fisik untuk membaca/menulis blok disk via bus VirtIO.

Struktur Inode pada Disk (struct dinode)

```
struct dinode {  
    short type;           // Tipe file (File, Directory, Device)  
    short major;          // Major device number (khusus device)  
    short minor;          // Minor device number (khusus device)  
    short nlink;          // Jumlah hard links  
    uint size;            // Ukuran file dalam bytes  
    uint addrs[NDIRECT+1]; // Pointer blok data (12 Direct + 1 Singly-Indirect)  
};
```

Kapasitas Maksimum Berkas Bawaan xv6

- 12 Direct blocks: $12 \times 1024 \text{ B} = 12 \text{ KB}$
- 1 Singly-Indirect block: $256 \times 1024 \text{ B} = 256 \text{ KB}$
- Total maksimum: 268 KB (Dapat ditingkatkan dengan Doubly-Indirect Blocks).

Crash Recovery dengan Write-Ahead Logging

Masalah Crash & Ketidakkonsistenan Disk

Jika sistem mati saat membuat berkas baru (misal: inode sudah ditulis tapi pointer direktori belum), sistem berkas akan rusak (*corrupted*).

Solusi Jurnalng xv6

```
begin_op(); // Mulai transaksi atomik
... // Lakukan modifikasi metadata di buffer cache
log_write(b); // Tandai blok untuk ditulis ke Log Header
end_op(); // Tulis log ke disk, commit, lalu instal ke lokasi asli
```

Jika crash terjadi, saat booting kernel membaca log dan memutar ulang (*replay*) seluruh transaksi yang belum selesai!

Ada Pertanyaan?

Praktikum: Buka `worksheet14.pdf` untuk eksplorasi modul `fs.c` dan `log.c`.